Traffic Sign Detection and Classification Using Real Road Images

Mannat Varshney | Revised 28 October 2026

Abstract

This project develops a five-category traffic-sign detection and classification system using only the Mapillary Traffic Sign Dataset (MTSD). Real annotated crops compare pretrained ResNet-18, MobileNetV3-Small and EfficientNet-B0. Full road scenes train a lightweight YOLO11n detector. Both tasks inherit the same protected scene split: 2,018 training, 356 validation and 332 local test scenes. The selected classifier achieves 96.94% accuracy on 425 annotated test crops. Detector mAP@0.50 is 0.4839 and mAP@0.50:0.95 is 0.3344. Actual ONNX inference is checked. Crop accuracy, scene detection quality and processing speed are reported separately.

1. Introduction and Motivation

Traffic-sign inspection is a practical computer-vision task: a road photograph may contain a small, blurred or partly obscured sign among buildings, vehicles and other objects. The objective is to locate supported signs and label them as stop, no entry, yield, speed limit 30 or speed limit 50. The tool saves annotated images or video and a detection table. It is not a driving controller and does not decide whether a road is safe.

The applied problem is finding small signs within complex road photographs. The project tests whether lightweight models and overlapping inference tiles provide useful recognition and localisation under limited computing resources. It saves annotated images/video and prediction tables. It does not claim a complete road-inventory service or measured operational time savings.

The project retains the Lightning/Hydra classifier structure and uses one dataset in two forms: complete photographs for detection and their annotated sign regions for classification. Classification answers which category a supplied rectangle contains; detection must also find that rectangle. MTSD supplies both real scenes and manually annotated boxes [2]. No synthetic training images or second traffic-sign dataset are used in the final experiments.

1.1 Research Questions

The experiments ask: which small classifier performs best under the same training budget; how detector input resolution changes validation detection quality; whether a separately trained crop classifier helps or hurts on detected global road-scene crops; and whether exported ONNX inference preserves the measured detector results. Model selection uses validation data, not final test scores.

1.2 Related Work and Project Contribution

GTSRB and GTSDB motivate the distinction between recognition and detection [1]. The MTSD authors study globally varied signs and explicitly provide full scenes with fine-grained labels [2]. DFG demonstrates traffic-sign inventory as an application [3]. This project uses those ideas for a bounded student implementation: a five-class detector, controlled crop baselines, an executable image/video interface, deployment verification and reproducible experiment records. Published paper scores are not reported as this project's results.

Two additional repositories were reviewed during finalisation [5,6]. The first illustrates webcam detections; the second separates detection from crop recognition. Their class counts, datasets and budgets differ. Their reported scores are not comparable baselines here, and no code or weights were imported. Detailed comparison: report/reference_projects.md.

2. Methodology

2.1 Real Data, Annotations and EDA

The classifiers use 2,563 training crops, 445 validation crops and 425 test crops cut from the prepared MTSD scenes. Crop splits are inherited from source photographs and similarity groups, never assigned independently. Each crop retains source-image identity, rectangle, category and hash. Training crop counts are stop 498, no entry 546, yield 576, speed 30 478 and speed 50 465. Resizing a small real crop does not create new image detail.

For detection, the authorised MTSD v2 fully annotated release is used. Its annotation ZIP matches the supplied MD5. Original selected image members pass ZIP CRC32 and have recorded SHA256 digests. Range downloads retrieve only the selected images; full image-archive MD5 verification is not claimed. No signs are drawn or pasted into generated scenes.

Category	Train scenes	Val scenes	Test scenes	Train boxes
stop	433	77	71	498
no entry	458	89	71	546
yield	478	84	81	576
speed limit 30	436	65	70	478
speed limit 50	427	73	70	465

One scene can contain several boxes or categories, so category scene counts do not sum to total scenes. More than 400 distinct training scenes per class remain after validation is reserved; this is not 500 independent physical signs per class. Selected official validation scenes form the final local test; hidden test labels are unused. Global speed classes identify the posted number and sign family, not an inferred or converted national speed unit.



Figure 1. Actual MTSD training photograph. Yellow boxes mark only the five selected categories. Large nearby road-bump and oncoming-traffic-priority signs are annotated in the source but outside this task. These are ground-truth boxes, not detector predictions.

2.2 Split Protection and Exclusions

Exact decoded-pixel checks and 64-bit full-scene difference-hash components screen visually similar images. Three training scenes in components overlapping held-out data are removed. Internal training and validation are split by those groups. This screening does not prove physical-sign independence without sequence metadata. Panoramas and scenes with selected target signs flagged ambiguous, occluded or out-of-frame are excluded. The scope is therefore easier than unrestricted driving footage.

Following annotation review, all 2,706 prepared scenes and 3,433 target boxes were compared with the five exact g1 source labels and coordinates: 0 scene mismatches were found. Other source labels are background. This includes 19 related regulatory stop/speed variants and one complementary speed sign not merged in the recorded task. Thus the five names do not imply all international variants are covered. Broader coverage requires reviewed remapping and retraining. The audit verifies mapped-label retention, not completeness of the original annotations.

2.3 Models, Preprocessing and Training

The classifier study fine-tunes all parameters of three ImageNet-pretrained models for five epochs on Apple MPS, using 128-pixel RGB crops, ImageNet normalisation, AdamW, learning rate 0.0003, weight decay 0.0001, batch size 16 and seed 42. Training uses small rotations and colour jitter without horizontal flipping. Checkpoints and architecture are selected by validation macro-F1. Test scoring and forward latency use CPU. ImageNet/COCO initial weights are transfer learning, not additional project traffic-sign datasets.

YOLO11n starts from general pretrained weights with a new five-class head. Initial CPU runs compare 416/640 inputs for five epochs, batch 8, AdamW, learning rate 0.001, weight decay 0.0005 and seed 42; ten backbone blocks are frozen. Real scenes receive small rotation, translation, scaling and colour changes, not flips, mosaic or copy-paste. Prepared scenes preserve aspect ratio at up to 1,280 pixels, then are letterboxed for training.

A further 15-epoch run starts from the selected 640 checkpoint, with five frozen blocks, learning rate 0.0005 and Apple M1 MPS. This validation-led extension changes initialization, budget and hardware; durations are not a controlled speed comparison.

Small objects are addressed using SAHI overlapping 640-pixel inference tiles, including a whole-image pass, 20% overlap and class-aware NMS at IoU 0.7. Whole versus tiled mode is selected on 356 internal validation scenes using mAP50:95. A confidence of 0.343 is selected by validation F1. Tiling changes inference, not training-image counts, and cannot reconstruct detail lost in the 1,280-pixel preparation.

2.4 Metrics and Their Selection

Crop accuracy is the fraction correct. Precision measures how often a predicted category is correct; recall measures the fraction of actual members found. F1 = 2PR/(P+R). Macro-F1 weights unequal-support categories equally and selects the classifier. Confusion matrices expose individual category errors.

Detector IoU is box-intersection area divided by union area. mAP50 averages class average precision at IoU >=0.50; mAP50:95 averages ten IoU thresholds, tests tighter localisation and selects detector settings. Unlike crop accuracy, these scores account for misses and false detections. Fixed-threshold P/R/F1 describe one operating point.

3. Experiments and Results

3.1 Mapillary Classifier Comparison

Model	Val F1	Test acc.	Macro P	Macro R	Macro F1
ResNet-18	0.9619	96.94%	0.9695	0.9686	0.9687
MobileNetV3-Small	0.9463	95.76%	0.9564	0.9551	0.9557
EfficientNet-B0	0.9477	96.47%	0.9643	0.9634	0.9633

ResNet-18 is selected by validation, not test accuracy. It correctly predicts 412 of 425 annotated test crops. The saved confusion matrix and per-image predictions identify the errors. The majority-training-label baseline scores 23.53% accuracy. Parameter counts and measured CPU forward times are retained with results. These are not scene-detection accuracy scores.

3.2 Controlled Detector Comparison

Run	Val mAP50	Val mAP50:95	Train min
yolo11n_416	0.1640	0.1049	11.5
yolo11n_640	0.2885	0.1943	24.6
yolo11n_640_finetuned	0.3398	0.2302	36.9

Selected before test evaluation: yolo11n_640_finetuned. The two initial resolution runs share a budget and split. The extra fine-tune has a different initialization and budget. These are settings of one detector architecture, not three architectures. Each is a single-seed run.

Inference mode	Val mAP50	Val mAP50:95	Mean ms/scene
whole	0.3209	0.2198	51.5
sliced	0.4859	0.3379	410.5

Same checkpoint, validation scenes and fixed-square preprocessing: tiling improves mAP50:95 from 0.2198 to 0.3379 at about eight times the read/detector cost. Training-validator preprocessing differs, so its scores are separate from this controlled mode comparison.

3.3 Held-out Road-Scene Results

On 332 selected held-out scenes, detector mAP@0.50 is 0.4839 and mAP@0.50:0.95 is 0.3344. Precision and recall at the maximum-F1 curve point are 0.6031 and 0.4653; they are not the same as the fixed-confidence numbers below.

Category	AP50	AP50:95
stop	0.6767	0.4849
no entry	0.3914	0.2415
yield	0.5804	0.3830
speed limit 30	0.3609	0.2608
speed limit 50	0.4102	0.3017

3.4 Detector Labels versus Crop Refinement

Pipeline	Precision	Recall	F1	TP / FP / FN
Detector only	0.5811	0.4635	0.5157	197 / 142 / 228
Detector + crop model	0.6726	0.5365	0.5969	228 / 111 / 197

These counts use validation-selected confidence 0.343, IoU 0.50 and one-to-one same-class matches. The new Mapillary-trained ONNX classifier changes labels, not boxes, and cannot recover misses. The predeclared application default remains detector-only; no test-driven switch is made. On 229 class-agnostically matched accepted boxes, refined crop accuracy is 99.56%. That conditional score excludes missed signs. Ground-truth crop accuracy and predicted-box accuracy have different inputs.



Figure 2. Actual processed held-out road scene. The demo selects early examples by source category, not correctness, and also contains missed or incorrect detections. A gallery video of these real still photographs is not continuous road footage.

3.5 Processing Time

The warm PyTorch-detector-plus-ONNX-classifier pipeline averages 435.35 ms per scene, with median 433.19 ms and 95th percentile 463.24 ms. Timing includes image reading, detector preprocessing/inference/NMS and crop inference; loading models, scoring and output encoding are excluded. Measurements use the recorded macOS CPU system, two PyTorch threads and two classifier ONNX threads. This is not a continuous-video real-time guarantee.

4. Deployment, Reproducibility and Discussion

4.1 ONNX and Optimisation

ONNX is an exported model format, not a third neural architecture. The selected detector is exported with opset 17, fixed batch size 1 and 640-pixel input. ONNX model structure checks pass. Raw outputs on three real test images are compared numerically with PyTorch; the largest absolute difference is 0.000793. Preprocessing and non-maximum suppression are still provided by the application, not embedded in the graph.

Actual ONNX detection on all 332 local test scenes gives mAP50 0.4839, versus PyTorch 0.4839. The new classifier float32 and dense-only dynamic INT8 exports both score 96.94% on 425 crops. CPU forward means are 10.15/10.11 ms (10 warm-ups, 50 repeats), with sizes 44.708/44.705 MB. No convincing speed/compression gain is established. Convolutions stay float32; this is not detector quantisation or pruning.

4.2 Versioning and Execution

The project uses Git for code, DVC for data/model versions, TensorBoard for training curves, Hydra for classifier configuration, and Lightning for the classifier loop. Detection settings are in pipelines/detection/. The image/video command writes annotated media, CSV detections and measured timing. Dataset terms apply separately from code; temporary authorised download links are not committed. Restoring licensed data requires authorised artifact access or regeneration from an authorised download.

Published Mapillary-only commit 449ef7a passes Docker build, container tests and two-frame ONNX inference in Actions run 40690970167. Full retraining inside Docker is unverified; later revisions require their own CI evidence. The submission is a fork of Vladimir-125/lightning-hydra-template with its MIT notice and history. No shared DVC remote is configured; full training needs authorised data/cache access.

4.3 Application and Software Verification

The command-line application accepts an image or video, runs whole/tiled detection, optionally classifies accepted crops, and saves annotated media, CSV detections and timing. Scores from the two models are kept separate. The packaged models are hash-checked after fresh extraction and tested on real inputs. Software tests check data handling, training, export and inference utilities; they do not prove predictive reliability. Visual inspection of original images remains necessary because signs are still missed.

4.4 Difficulties and Limits

The main difficulties were acquiring real scenes, protecting splits, preserving labels and finding small signs. The mentor identified apparently missing labels and tiny targets. Source comparison showed nearby omitted signs were unsupported categories; the reviewed speed-50 targets shrink to about 14 pixels at preparation and 7 at whole-scene input. Numbered views fix report readability; tiling measurably improves aggregate detection but does not solve every tiny sign. The final revision retrains all crop models on Mapillary instead of reusing earlier cross-dataset scores.

The study uses selected positive scenes, excludes difficult targets and uses one seed per run. Hashes do not prove physical-sign independence. The protected holdout was inspected in earlier project development, so this is not an external blind evaluation. Unsupported categories and false alarms on sign-free journeys remain unmeasured. The gallery is qualitative evidence, not a moving-video benchmark. More training, difficult scenes, repeated seeds and independent real-road evaluation are needed.

5. Conclusion

One Mapillary dataset supports the detection, crop-classification and exported-inference workflow. The study compares three classifiers and detector settings, saves actual output and verifies ONNX inference. The selected classifier scores 96.94% on 425 annotated crops; local detector mAP50 is 0.4839. These answer different questions. Small-sign detection remains imperfect; the study does not establish autonomous-driving reliability.

References and Evidence

[1] Stallkamp et al., GTSRB, IJCNN 2011; Houben et al., GTSDB, IJCNN 2013 (related work only, not project data). [2] Ertler et al., MTSD, ECCV 2020. mapillary.com/dataset/trafficsign. [3] Tabernik and Skocaj, IEEE T-ITS 2019. vicos.si/resources/dfg/. [4] Akyon et al., SAHI, ICIP 2022. github.com/obss/sahi. [5] github.com/ayushgayakwad/road-sign-recognition. [6] github.com/pedramebd/traffic-sign-detection-recognition. Repositories reviewed 28 October 2026. Evidence: report/mtsd_classifier_results/, mtsd_results/, mtsd_onnx_results/, mtsd_annotation_review/.